

Lab 5: Power Analysis

PSYC 6802

Fall 2024

Packages for Today (Install If Needed)

```
library(tidyverse)  
library(pwr)
```

Overview

The primary package we'll use for conducting power analyses is `pwr`.

Functions we will be using today:

- `pwr.norm.test`
- `pwr.t.test`
- `pwr.t2n.test`
- `pwr.chisq.test`

The Logic of the pwr Package Functions

Most of the functions of the `pwr` structure work in the same way in that they usually have four main arguments:

- The effect size, `d` =, in the case of *t*-tests.
- The sample size, `n` =
- The type I error rate, `sig.level` =
- The power of your experiment, `power` =

The `pwr` functions always return the value of the one argument that you leave blank out of these four.

- These arguments may be named differently depending on the type of power analysis, so always check the function help menu.

Power for z-tests

In the case of a z-test, to get **power** given effect size, sample size, and desired type I error rate:

```
pwr.norm.test(d = .5,  
              n = 10,  
              sig.level = .05,  
              power = NULL,  
              alternative = "two.sided")
```

Mean power calculation for normal distribution with known vari

```
      d = 0.5  
      n = 10  
sig.level = 0.05  
  power = 0.3526081  
alternative = two.sided
```

Power for z-tests

In the case of a z-test, to get required **sample size** given effect size, desired type I error rate, and power:

```
pwr.norm.test(d = .5,  
              n = NULL,  
              sig.level = .05,  
              power = .8,  
              alternative = "two.sided")
```

Mean power calculation for normal distribution with known vari

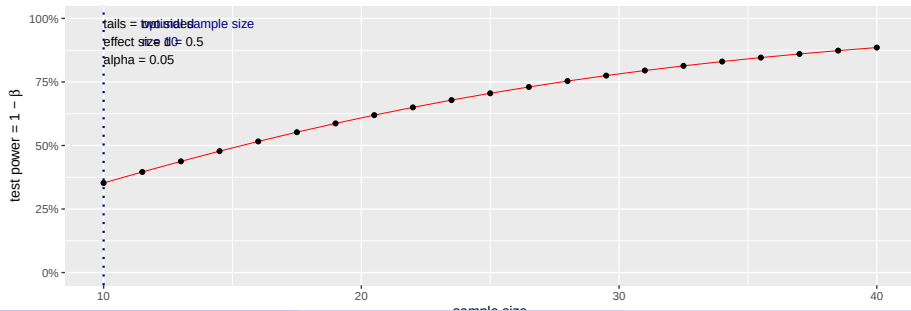
```
      d = 0.5  
      n = 31.39544  
sig.level = 0.05  
  power = 0.8  
alternative = two.sided
```

Power for z-tests

The `pwr` package also includes a plotting method. This means that we can use the function `plot()` with the output from any power calculation results. (they don't look super good)

```
plot(pwr.norm.test(d = .5,  
  n = 10,  
  sig.level = .05,  
  power = NULL,  
  alternative = "two.sided"))
```

Mean power calculation for normal distribution
with known variance



One-sample t -test

In lecture 4 we ran a one sample t -test to test whether average restaurant ratings were significantly different from 3. with $d = .21$ and $N = 200$ we can do a retrospective power calculation:

```
pwr.t.test(n = 200,  
           d = .21,  
           sig.level = .05,  
           power = NULL,  
           type = "one.sample",  
           alternative = "two.sided")
```

One-sample t test power calculation

```
      n = 200  
      d = 0.21  
sig.level = 0.05  
  power = 0.8402593  
alternative = two.sided
```

Our power was around .84. What does this mean?

One-sample t -test

What if I want to repeat the previous “experiment”, and I want to know the sample size that I would need to have .8, .9, and .95 power?

```
pwr.t.test(n = NULL, d = .21, sig.level = .05, power = .80,  
           type = "one.sample", alternative = "two.sided")
```

```
pwr.t.test(n = NULL, d = .21, sig.level = .05, power = .90,  
           type = "one.sample", alternative = "two.sided")
```

```
pwr.t.test(n = NULL, d = .21, sig.level = .05, power = .95,  
           type = "one.sample", alternative = "two.sided")
```

For a priori power analysis, we want to round up, so $N = 180, 240, 297$ for 80%, 90%, and 95% power.

Power analysis for a paired-sample t -test is the same.

Independent-samples t -test

For the independent samples t -test, we can use `pwr.t.test` if we have equal sample sizes, or `pwr.t2n.test` for unequal sample sizes.

In the case of lab 4, where we were testing whether males ($n_1 = 112$) and females ($n_2 = 82$) were significantly different on average food rating. the estimate effect was $d = .17$, but the difference was not significant $p = .224$:

```
pwr.t2n.test(n1 = 112, n2 = 82, d = .17, sig.level = .05,  
             power = NULL, alternative = "two.sided")
```

```
t test power calculation
```

```
      n1 = 112  
      n2 = 82  
      d = 0.17  
sig.level = 0.05  
  power = 0.2138687  
alternative = two.sided
```

In retrospect, their power to detect an effect was only .21!

Independent-samples t -test

When planning data collection, you ideally want to plan for groups to have an equal size if possible. So, we can calculate a priori power assuming equal sample sizes and $d = .17$ as an effect size estimate.

```
pwr.t.test(n = NULL, d = .17, sig.level = .05, power = .80,  
           type = "two.sample", alternative = "two.sided")  
pwr.t.test(n = NULL, d = .17, sig.level = .05, power = .90,  
           type = "two.sample", alternative = "two.sided")  
pwr.t.test(n = NULL, d = .17, sig.level = .05, power = .95,  
           type = "two.sample", alternative = "two.sided")
```

It turns out that $n = 545, 729$, and 901 (quite large!) are the minimum sample sizes **in each group** for 80%, 90%, and 95% power.

Power for χ^2 tests

We can do the same for χ^2 tests using the `pwr.chi.test` function. The test does not distinguish between χ^2 goodness-of-fit and χ^2 test of independence.

Let's use the airplane delays example from Lab 3. $\chi^2(1) = 410.66$ with $N = 106596$. Our W coefficient was .06.

```
pwr.chisq.test(w = .06, N = 106596, df = 1, sig.level = .05, power = NULL)
```

Chi squared power calculation

```
w = 0.06  
N = 106596  
df = 1  
sig.level = 0.05  
power = 1
```

NOTE: N is the number of observations

Power is essentially 1, we are guaranteed to always reject the null hypothesis under these circumstances.

Power for χ^2 tests

Next, let's conduct a priori power for this study. Output not shown.

```
pwr.chisq.test(w = .06, N = NULL, df = 1,  
               sig.level = .05, power = .80)  
pwr.chisq.test(w = .06, N = NULL, df = 1,  
               sig.level = .05, power = .90)  
pwr.chisq.test(w = .06, N = NULL, df = 1,  
               sig.level = .05, power = .95)
```

For this effect size, we would just need $N = 2181, 2919, 3610$ for 80%, 90%, and 95% power.

Section 1

My .Rmd file Won't Knit :(

Let's work together

I have created a .Rmd file that contains errors that will cause it not to knit. Download the `Assignment_2_PSYC_7965.Rmd` file and let's try to solve the errors together until the file knits!

- We will also need the `sat_data.csv` dataset to run the code.

The following slides are not very detailed, as the process of solving knitting issues varies greatly, but you can watch the recording for Lab 5 if something is not very clear.

Step 1: Make sure R code works

Before you even attempt knitting it is good practice to clean your environment and run all your R code chunks sequentially.

- To quickly run all the code chunks either navigate to the run option in the toolbar to the right of the source pane and click Run All, or use the shortcut CMD + Option + R on MAC or CTRL + Alt + R on Windows

Let's solve the code errors until all the chunks run sequentially.

Step 2: Knit and solve LaTeX errors

If all your R chunks run fine and your document still does not knit, there is probably some problem with your text, which is a LaTeX related problem.

Whenever your Rmarkdown files fails to knit, a **.tex** file and a **.log** file will appear. **.tex** files contain the code that LaTeX reads to create PDFs.

- The file will stop knitting because the LaTeX interpreter has found some code that it does not understand in the **.tex** file.
- The **.log** file, among other things, specifies the **.tex** line that caused the error. R will also show you the line that caused the error.

What you need to do is find the line in your **.Rmd** file that corresponds to the line that caused the error in the **.tex** file and solve the problem.